

Högskolan i Gävle



Kartläggning av serverhallar och el-centraler

Projektarbete

Ilayda Gül

llaayddaaaa@gmail.com

Andre Camacho

andrecamare01@gmail.com

Vimbainaishe P Vambe

vpvambe@gmail.com

2025-05-22

Innehållsförteckning

Inledning.....	3
Datastrukturer.....	3
Algoritmer.....	4
Tester.....	5
Resultat	7
Utvärdering.....	8
Referenser	9

Inledning

Denna rapport presenterar design, implementation och utvärdering av en Javabaserad prototyp som utvecklats för att kartlägga svenska serverhallar och el-centraler. Det primära målet var att designa ett verktyg som kan hantera och analysera stora mängder positionsdata, etablera en grafstruktur för att representera serverhallar, beräkna kortaste vägar mellan dem, samt erbjuda ett användarvänligt gränssnitt för visualisering.

Prototypen har även möjlighet att simulera och jämföra uppdaterade GPS positioner med befintliga data, vilket ger stöd för att upptäcka förändringar och säkerställa datakvalitet. En central funktion i prototypen är beräkningen av kortaste vägar med hjälp av Dijkstras algoritm, tillsammans med ett användarvänligt gränssnitt (GUI) för att visualisera nätverket och underlätta fältarbete och infrastrukturplanering.

Utöver de funktionella målen fokuserar projektet även på att analysera körtidseffektiviteten hos Dijkstras algoritm när den tillämpas på små grafer med ett begränsat antal noder och kanter. Särskild uppmärksamhet ges åt hur olika designval såsom datastrukturer som används för att representera grafen, påverkas algoritmens prestanda. Rapporten innehåller tidstestet och utvärderingar som ger insikter i effektiviteten och lämpligheten hos de implementerade lösningarna.

Datastrukturer

Grafen är implementerad med noder och kanter där varje nod underhåller en lista över sina anslutna kanter. Detta tillvägagångssätt valdes på grund av dess minneseffektivitet i glesa grafer, vilket är typiskt för detta problemområde med upp till fem noder och en eller två kanter per nod.

Varje nod innehåller koordinatinformation (för visualisering) samt referenser till sina kanter, vilket är inkapslade i en Edge-klass som innehåller målnoden och kantens vikt. Den övergripande grafstrukturen använder en Hashmap som ger enkel åtkomst till noder som ger $O(1)$ tidskomplexitet. Vid användningen av Dijkstras algoritm kontrollera vilka noder som finns och vissa specifika noder och detta är effektivt med Hashmap. Nycklarna i hashmap är unika vilket säkerställer att det inte finns dubletter av noder. Gällande kanterna som finns i grafen används också hashmap för att hålla koll på vilka kanter som är kopplade till en viss nod. Varje nod har ett ID och värdet är listan över utgående kanter från just den noden. Det

går snabbt att hitta nodens grannar med hashmap. ArrayList är effektivt att använda när varje nod kan ha ett varierande antal grannar och man kan lägga till snabbt med en ArrayList eftersom den är mer dynamisk. Dessutom är det enkelt att rita grafen i gränssnittet när man ska iterera över alla grannar till varje nod med en ArrayList. Användningen av en kantlista minskar minnesanvändningen avsevärt jämfört med intillighetsmatrisen, för små, glesa grafer, vilket gör denna struktur idealiskt för applikationen.

Prioritetskö är bland de viktigaste komponenterna för att implementera Dijkstras algoritm på ett effektivt sätt. Vid denna algoritm behöver den kontinuerligt hålla reda på den nod som har det kortaste avståndet från startnoden. Vid varje iteration måste algoritmen plocka ut den mest lämpliga noden med lägst vikt hittills och det kan en prioritetskö göra på ett effektivt sätt logaritmiskt. I implementationen är det när vi väl hittar det kortaste avståndet till en granne uppdateras avståndet och läggs i kön och detta säkerställer att varje nod behandlas i ordning som följer stigande avstånd från startpunkten till slutpunkten.

För att mäta skillnaden mellan de ursprungliga och uppdaterade positionerna i grafen används Hausdorff-avståndet. För att göra beräkningarna snabba och effektiva används en Hashmap som kopplar varje nod till dess nya position, vilket gör att sökningar sker på konstant tid, $O(1)$. Positionerna sparas i enkla objekt med x och y koordinater, vilket underlättar beräkningar av avstånd och visning i grafiken. Eftersom algoritmen jämför alla punkter mot varandra blir tidskomplexitet ungefär $O(n^2)$. För större datamängder kan man använda metoder som Quadrees för att minska antalet jämförelser och förbättra prestandan. Den här lösningen ger en bra balans mellan minnesanvändning och beräkningshastighet, vilket är viktigt för att kunna hantera större grafer effektivt. Hausdorff-avståndet visar dessutom tydligt vilken nod som har störst positionsskillnad, vilket hjälper till vid analys och visualisering.

Algoritmer

Den centrala algoritmen som testas är Dijkstras algoritm för att hitta den kortaste vägen från en startnod till andra i en viktad graf samt övriga kanter i en oriktad graf där alla kanter är icke-negativa vikter.

Hur Dijkstras algoritm fungerar är att man till en början sätta avståndet på startnoden till 0 och alla andra noder till oändligheten. Läger sedan alla noder i en prioritetskö som kontinuerligt sorterar noderna baserat på deras hittills kortaste avstånd. För att sedan iterativt hämta den noden med kortast avstånd från kön. Sedan uppdateras deras grannars avstånd om det finns en kortare väg via den specifika noden och detta upprepas tills att alla noder har besökts eller att slutmålet har besökts.

Enligt GeeksforGeeks (2023) har algoritmens tidskomplexitet $O((V + E) \log V)$, där V är antalet noder och E är antalet kanter ifall man använder en prioritetskö där varje poll operation på en nod och varje uppdatering på en kant ger tidskomplexiteten $\log V$ som är logaritmisk per operation.

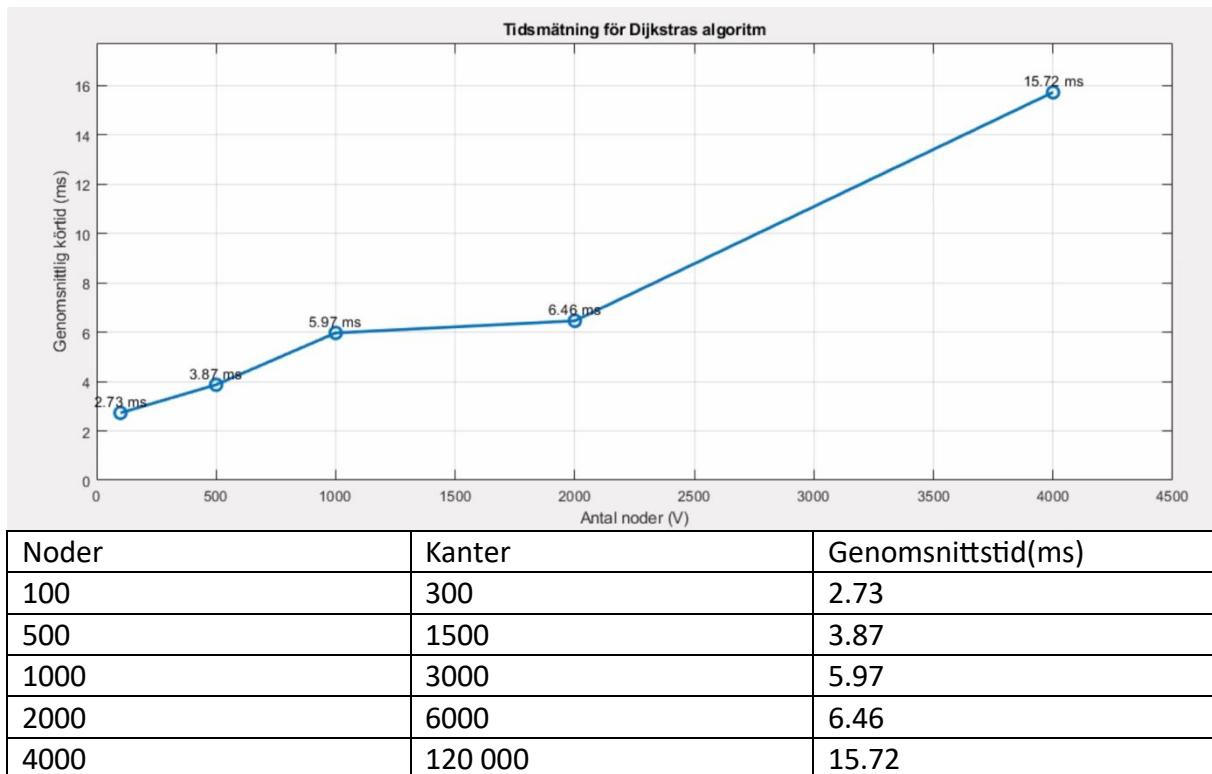
Givet de små grafstorlekarna körs algoritmen effektivt och nästintill i realtid.

Utöver Dijkstras algoritm har även beräkning av Hausdorff-distansen implementerats för att mäta skillnaden mellan två grupper av punkter, i detta fall koordinater för serverhallar, både deras ursprungliga positioner och uppdaterade positioner. Hausdorff-distansen anger det största avståndet från en punkt i den ena gruppen till närmaste punkt i den andra gruppen. Det gör det enkelt att se hur mycket de uppdaterade positionerna skiljer sig från de ursprungliga. Denna beräkning är viktigt för att mäta hur noggranna positionerna är efter en uppdatering. Den hjälper också till att hitta den nod som har störst fel, vilket är användbart när fokus ska läggas på att rätta till problem där felet är som störst. För att beräkningen snabbt har datastrukturer som hashmaps och listor valts, vilka gör det lätt att hitta rätt punkter snabbt. Resultaten från Hausdorff beräkningen används också i visualiseringen för att tydligt visa vilken nod som har störst avvikelse.

Andra algoritmer, såsom bredden-först-sökning (BFS) eller Bellman-Fords algoritm, övervägdes men ansågs onödigt för detta problem med viktade grafer.

Tester

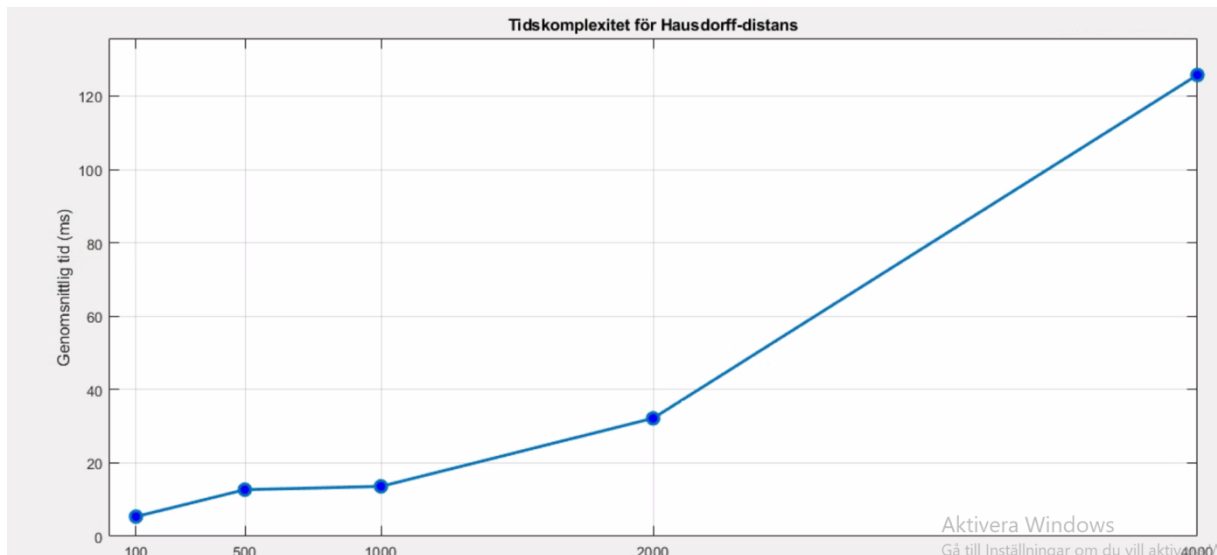
Dijkstras Algoritm



När man tittar på den teoretiska förväntan för Dijkstras algoritm i en graf har den tidskomplexiteten $O((V + E) \log V)$ där V är antalet noder och E är antalet kanter.

Algoritmen kördes på genererade grafer som slumpar antalet noder och kanter där vi satte att varje nod har tre vägar att välja mellan vilket resulterar E multiplicerat med 3. När man multiplicerar noder med 3 så varje nod har 3 kanter så har den tidskomplexiteten $V \log V$. I iterationen mättes körtiden i millisekunder fem gånger och tog ut ett medelvärde för genomsnittstiden. Grafen visar en stabil ökning mellan 100 till 2000 noder och med en tydligare ökning upp till 4000 noder. Detta stämmer överens med den teoretiska förväntan för $V \log V$ eftersom den växer långsamt för små och medelstora grafer. Däremot vid 4000 noder får prioritetskön fler operationer att arbeta med vilket tar betydligt längre tid.

Hausdorff Distansen



Noder	Genomsnittstid(ms)
100	5.37
500	12.71
1000	13.61
2000	32.17
4000	125.75

Hausdorff-algoritm säger att varje punkt i delmängd A ska jämföras med varje punkt i delmängd B och tvärtom, detta är anledningen till att den algoritmen har tidskomplexitet $O(n^2)$. Det är n som är antalet datapunkter i varje delmängd där två riktningar kontrolleras från A till B och tvärtom.

I de genomförda mätningarna visar att från 100 till 500 datapunkter ökar tiden från 5.37 ms till 12.71 milisekunder vilket är mer än fördubbling av tid. Sedan ökar det inte lika mycket med tid från 12.71 till 13.61 mellan 500 till 1000 datapunkter. Däremot från 1000 till 4000 element ökar det fyra gånger mer och detta visar ett mönster som är kvadratisk tidskomplexitet som är den förväntade tidskomplexiteten $O(n^2)$.

Resultat

Resultatet av tidsstesterna visade det att exekveringstiden ökade både för Hausdorff distansen och Dijkstras algoritmen beroende på antalet av noder och kanter. För Dijkstras algoritmen sker det en ökning relativt linjärt för ökningen med antalet noder upp till 2000 noder och sedan vid 4000 noder ser man en tydligare ökning. Detta stämmer överens med den teoretiska tidskomplexiteten för Dijkstras algoritm som är $O((V+E) \log V)$ och med 3E blir tidskomplexiteten $O(V \log V)$.

Resultatet för Hausdorff distansen visade att förväntade tidskomplexiteten var kvadratisk tillväxt eftersom varje punkt jämförs med en annan punkt. Varje punkt i delmängd A ska jämföras med varje punkt i delmängd B som ger $O(n)$ men eftersom jämförelsen sker två gånger blir tidskomplexiteten $O(n^2)$ som är kvadratisk tidskomplexitet. Detta visas tydligt i grafen att tiden fördubblas när den går från 2000 till 4000 noder. Grafen visualiseras med linjediagram i MATLAB för att visa mönster och tillväxt för exekveringstiden med dessa algoritmer.

Utvärdering

Applikationen uppfyller sitt mål och syfte med att skapa en grafstruktur som visualiserar noder och kanter som ska representera serverhallar och vägar. Detta är för att beräkna den kortaste vägen från startnod till alla andra noder med hjälp av Dijkstras algoritm samt göra GPS-förflyttningar och visa Hausdorff distansen på ett interaktivt sätt. Det finns även speciella funktioner i grafen som att zooma in och ut för att göra grafen mer luftig. Man kan hitta informationen om varje nod om man trycker på den samt ändra färg på noderna i olika teman samt visualisera en bil när man beräknar kortaste vägen. Dessutom att exportera som bild som förvaras i mappstrukturen i Eclipse.

Tidsmätningarna för Dijkstras algoritm presterar enligt den teoretiska förväntan vilket är $O(V \log V)$ för noder som har max 3 kanter vilket grafen visar gällande ökning mellan 1000 noder. Hausdorff-distansten visade kvadratisk tillväxt vilket visade tydligt i grafen vid exekveringen mellan 2000 till 4000 noder och där tiden fördubblades.

Designvalet var att implementera en kantlista som är en hashmap som är effektivt för glesa grafer och det är enkelt att iterera över dess grannar även med en ArrayList. Detta gör att det enkelt att lagra vikter och koordinater som är viktigt att veta vid dessa typer av problem.

Förbättringsområden skulle kunna vara att användargränssnittet tillåter inte lägga till och ta bort nod samt kunna redigera en nod eller en kant direkt i gränssnittet. Det finns heller inte information om nodens vägar och på detta sätt kan användaren enkelt veta hur många vägar och vilka kopplingar den har och information om dess granne. En funktionalitet som skulle kunna läggas till är att jämföra olika rutter med varandra gällande tiden samt visning av tiden hur lång tid det tog att ta sig från nod A till nod B.

För framtida arbete skulle tester på större och mer komplexa grafer kunna ge insikt i skalbarheten. Att implementera avancerade datastrukturer, såsom ex Fibonacci-heapar, skulle även kunna optimera Dijkstras körtid för tätare grafer.

Referenser

GeeksforGeeks. (2022, 13 Oktober). *Dijkstra's shortest path algorithm in Java using PriorityQueue*. <https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-in-java-using-priorityqueue/>